Laboratory Report: Personal MP3 Player Using RT-Spark, STM32F407, and FreeRTOS

Name: Mary Grace V. Soronio

Subject: BCA182 Embedded Systems Programming

Year and Section: BSCA 4 - B187.1

Date: October 28, 2026

1. Abstract

This laboratory report presents the design, implementation, and testing of a Personal MP3 Player built on the RT-Thread RT-Spark development board equipped with an STM32F407 microcontroller. The system uses PlatformIO, STM32Cube HAL, and FreeRTOS to create a multitasking embedded audio player capable of playing multiple musical tones, adjusting volume, updating an LCD display, and responding to user input through hardware buttons and a potentiometer.

The player functions as a realistic embedded audio demonstration project. It generates musical notes in software, streams them through the ES8388 audio codec using I2S communication, and uses the onboard LCD to show the current song, playback state, and volume. Buttons are used for song selection, play/pause control, and tempo modification, while a potentiometer allows analog volume control. The software is structured as multiple concurrent tasks coordinated by FreeRTOS timers, semaphores, and mutexes.

This report documents the motivations behind the design, the technical architecture, the hardware and software configuration, the system operation, and the validation results obtained through build and upload testing. The report also includes the implementation details of the song selection logic, timing control, and the embedded firmware design used to realize the project.

2. Introduction

Embedded systems continue to play a major role in modern electronics because they combine real-time processing, compact hardware, and reliable software control. In educational and practical settings, microcontroller-based designs are often used to demonstrate the integration of signal processing, digital communication, user interfaces, and multitasking. The development of a portable music player using an STM32 microcontroller is a suitable application because it incorporates several important embedded systems concepts, such as GPIO control, ADC measurement, I2C and I2S communication, hardware timers, and real-time scheduling.

The RT-Spark board is an STM32F407-based development platform designed to support experimentation with embedded technologies. It includes multiple peripherals that make it appropriate for interfacing with audio hardware, user controls, and display modules. This project leverages these capabilities to implement a personal MP3-style player that is both educational and functional.

The objective of the project is not to create a full-featured MP3 decoder or a commercial media player. Instead, the design focuses on embedded audio synthesis and playback using a tone-generation approach. The system generates tones corresponding to musical notes, streams them through an audio codec, and provides a simple yet effective interface for user interaction. This makes the project highly relevant to embedded systems courses, as it demonstrates practical hardware control and system design under real-time constraints.

3. Project Background and Motivation

The project is inspired by the concept of a digital audio player that can be implemented directly on a microcontroller without the use of general-purpose operating systems or full desktop software frameworks. In embedded systems education, such projects are valuable because they require students to understand the interaction between hardware and software at a deep level.

A personal MP3 player, even in simplified form, requires:

? a method for generating or reading audio data,

? a digital interface for sound output,

? a way to receive and process user commands,

? a display for status information,

? timing and scheduling for smooth playback,

? and coordination among several hardware components and tasks.

This project achieves these goals using a real-time operating system and hardware peripheral drivers. It demonstrates how multiple software tasks can run concurrently while preserving system stability. For instance, one task may read inputs, another may adjust the volume, and another may refresh the LCD. A separate audio task configures the codec and begins the stream, while software timers regulate note timing and confirmation delays.

4. Problem Statement

A microcontroller-based audio player must provide interactive playback control, storage or generation of musical notes, and reliable real-time output. However, implementing these functionalities on a resource-constrained embedded system introduces several technical challenges:

1. Audio data must be generated or managed efficiently to avoid jitter and buffer underflow.
2. Button inputs must be debounced and interpreted correctly.
3. A visual interface must reflect the current state of the player.
4. Different components such as the codec, display, and input controls must be coordinated in a safe and deterministic manner.
5. The software must manage concurrency without causing display flicker, timing errors, or invalid audio output.

This project addresses these challenges by using FreeRTOS to organize the embedded system into manageable tasks and by using hardware peripherals to monitor user input and output sound.

5. Objectives

The primary objectives of this project are as follows:

1. To design and implement a portable embedded MP3-style player on the RT-Spark board.
2. To use STM32F407 hardware peripherals including GPIO, ADC, UART, I2C, and I2S.
3. To configure and control the ES8388 audio codec for audio output.
4. To generate musical tones through software synthesis and DMA-based I2S streaming.
5. To provide interactive user control for song selection, playback, tempo, and volume.

6. To display operational status information on the LCD and via UART.
7. To demonstrate the use of FreeRTOS tasks, timers, semaphores, and mutexes in an embedded system.
8. To assess the reliability and performance of the system through build, upload, and functional checks.

6. Scope and Limitations

6.1 Scope

The project includes:

? audio note generation using a software waveform model,
? processing of real-time user input,
? selection of pre-defined songs,
? LCD status display,
? volume adjustment using ADC,
? play/pause and tempo control,
? UART startup output,
? and multitasking coordination using FreeRTOS.

6.2 Limitations

The system is intentionally designed as a simplified educational implementation rather than a commercially complete media player. Some limitations include:

? the audio is generated from synthesized tones rather than full MP3/WAV decoding,
? songs are predefined in the firmware and not loaded from external storage,
? the available song list is limited to the provided course song definitions,
? hardware control is constrained by the onboard keys and periphery of the RT-Spark board,
? and the system is optimized for demonstration and academic study rather than consumer-grade audio quality.

7. System Overview

The Personal MP3 Player system is built around the RT-Spark board and consists of several key functional blocks:

1. Input block: four onboard keys, USER_BUTTON, and potentiometer.
2. Control block: STM32F407 microcontroller running FreeRTOS.
3. Audio block: ES8388 codec and I2S interface.
4. Display block: LCD screen for user feedback.
5. Indicator block: LEDs for playback status.
6. Communication block: UART serial monitor output.

These components work together to form a compact embedded application where user interaction and real-time audio processing are synchronized.

8. Hardware Design and Components

8.1 RT-Spark Board

The project uses the RT-Spark development board based on the STM32F407ZGT6 microcontroller. This device features a Cortex-M4F processor running at 168 MHz and provides rich peripheral support for embedded applications. The specific features used in the project include:

? GPIO pins for LEDs and keys,

? ADC for the potentiometer,

? I2C for codec control,

? I2S for audio streaming,

? USART for serial communication,

? and FSMC for controlling the LCD display.

8.2 Audio Codec

The ES8388 codec is used as the audio output device. It is connected via I2C2 for configuration and via I2S3 for the actual audio data stream. This allows the microcontroller to send digital audio data to the codec, which then outputs a stereo or mono analog signal to the headphone port.

8.3 Display Unit

The ST7789 TFT LCD with 240x240 resolution is connected to the STM32 via FSMC. It is used to display the currently selected song name, playback state, and volume. Because multiple tasks may need to update the LCD, a mutex protects the display resource to avoid concurrent access and visual corruption.

8.4 Input Controls

The system uses several user input sources:

? UP, LEFT, DOWN, and RIGHT buttons on the board,

? USER_BUTTON connected externally on the breadboard,

? potentiometer used to adjust volume,

? optional UART monitoring for debugging and instructions.

8.5 Indicators

Three LEDs indicate the state of the player:

? Yellow LED: playing state,

? Red LED: paused or stopped state,

? Green LED: song-selection confirmation mode.

9. Pin Mapping

Component	Pin/Interface
Yellow LED	PB14
Green LED	PA1
Red LED	PA8
Potentiometer (ADC3 Channel 4)	PF6
UP key	PC5
DOWN key	PC1
LEFT key	PC0
RIGHT key	PC4
USER_BUTTON	PA0
UART TX	PA9
UART RX	PA10
LCD data bus	PD14, PD15, PD0, PD1, PE7-PE10
LCD RS	PD13
LCD RD	PD4
LCD WR	PD5
LCD CS	PG10
LCD RST	PD3
LCD BL	PF9
Audio codec I2C SCL	PF1
Audio codec I2C SDA	PF0
I2S3 MCK	PC7
I2S3 WS	PA15
I2S3 CK	PB3
I2S3 SD	PB5

10. Software Tools and Environment

The project was developed using:

? PlatformIO as the embedded development environment,
? STM32Cube HAL for peripheral configuration and drivers,
? FreeRTOS for task scheduling and timing,
? C and C++ firmware sources integrated through a C wrapper,
? ST-LINK for flashing the board,
? and serial UART monitoring for debugging.

The project configuration in `platformio.ini` enables the STM32F407 board target, includes FreeRTOS header paths, and applies the required floating-point configuration for the Cortex-M4F processor.

11. Software Architecture

The firmware is organized around a multitasking model. Instead of placing all logic into one large loop, the system divides functions into independent tasks that work concurrently.

11.1 Main FreeRTOS Tasks

| Task | Responsibility |

| --- | --- |

| `update_lcd_leds_thread` | Refreshes the LCD display and updates LED states |

| `polling_buttons` | Monitors key presses and handles song selection logic |

| `adjust_volume` | Reads ADC input, updates volume, and prints UART instructions |

| `audio_task` | Initializes the codec and starts the audio stream |

11.2 Timing and Synchronization

The firmware uses FreeRTOS software timers for:

? note progression (Ticker),

? confirmation timeout for song selection,

? and timed state management.

A mutex (`lcd_mutex`) protects the LCD from concurrent access, and semaphores help coordinate shared resources and state updates. The idle task executes `WFI` (Wait For Interrupt) to reduce current consumption while the system waits for interrupts.

12. Methodology and Implementation

12.1 System Initialization

On startup, the firmware initializes the following subsystems:

? system clock configuration,

? GPIOs for LEDs and buttons,

? UART for serial communication,

? ADC for volume measurement,

? I2C for codec configuration,

? I2S and DMA for audio output,

? LCD driver for the display,

? and FreeRTOS tasks and timers.

The system then prints the song list and usage instructions through UART so the user can understand how to interact with the device.

12.2 Song Selection Logic

The project supports a binary selection method using the three code keys:

? LEFT = 4,

? DOWN = 2,

? RIGHT = 1.

The user holds one or more of the three keys to form a binary value from 000 to 111. A 5-second confirmation window is used before the selected song begins playing. If the user does not confirm, the system cancels the selection and retains the current song.

The binary value determines the target song index among the first eight songs defined in the course song table.

12.3 Audio Generation

The audio output is not stored as MP3 data. Instead, the firmware uses software-generated tones based on note periods in milliseconds. The period values are obtained from the course-provided song definitions and converted into a frequency-like signal. The digital waveform is generated and pushed into I2S DMA buffers.

The audio generation includes:

? note phase updates,

? buffer refill in half/full transfer interrupts,

? triangle-wave-like synthesis,

? tempo-based playback timing,

? and note rests represented by zero values.

This method allows the STM32 to stream audio continuously while remaining within the limitations of a low-resource embedded platform.

12.4 Volume Control

Volume is controlled by a potentiometer connected to ADC3 channel 4. The ADC value is read periodically, mapped to a percentage-like value, and applied to the codec output. The system thus allows direct user adjustment without needing additional software controls.

12.5 Menu and Status Display

The LCD shows the current song, player state, and volume. It displays a simple and readable status screen so the user can quickly determine whether the system is playing, paused, or waiting for a new song selection. The display update logic is synchronized to prevent overlap and to keep the output consistent.

13. Song List and Application Mapping

The project uses songs from the provided course definitions. The first eight songs are selectable through the binary code system.

| Code | Song |

| --- | --- |

| 000 | Fur Elise - Beethoven |

| 001 | Canon in D - Pachelbel |

| 010 | Minuet in G major - Bach |

| 011 | Turkish March - Mozart |

| 100 | Nocturne in E flat - Chopin |

| 101 | Waltz No. 2 - Shostakovich |

| 110 | Nocturne in C sharp minor - Chopin |

| 111 | Symphony No. 40 - Mozart |

14. Testing and Validation

14.1 Build Verification

The project was validated by building the firmware using PlatformIO. This confirms that the code compiles correctly with the embedded hardware configuration. The build setup includes the necessary include paths for FreeRTOS and the STM32F407 environment and applies required floating-point flags for the Cortex-M4F core.

14.2 Upload Verification

The firmware was uploaded through the ST-Link interface. This step ensures that the project is correctly configured for the RT-Spark board and that the board can receive flashed firmware.

14.3 Functional Verification

The main functional tests performed include:

? verifying that the LCD initializes properly,

? checking that the LEDs switch according to the player state,

? confirming play/pause toggling through USER_BUTTON,

? testing binary song selection using the code keys,

? confirming the 5-second confirmation timer behavior,

? testing tempo changes by holding the USER_BUTTON,

? verifying the potentiometer-driven volume response,

? confirming UART output during startup and operation.

14.4 Observed Behavior

When powered on, the system initializes hardware and prints user instructions to UART. The player is ready to receive a song selection. Once a valid song is chosen and confirmed, audio playback begins. The LED indicators and LCD output reflect the selected mode. The system remains responsive to input changes while preserving a stable timing structure for the audio stream.

15. Results and Discussion

The completed project demonstrates that a fully functional multimedia-like application can be implemented on a low-cost embedded microcontroller platform. The design integrates several layers of embedded engineering: hardware-level configuration, software multitasking, real-time timing, audio synthesis, and user interface control.

The use of FreeRTOS was particularly valuable because it allowed the firmware to organize responsibilities into independent tasks rather than forcing all logic into one blocking loop. This improved maintainability and made the behavior easier to reason about. The software timers were essential for controlling both note timing and user confirmation windows. This made the player responsive and reliable under real-time constraints.

The audio implementation proved to be a suitable educational method for teaching embedded signal generation, as it uses DMA and I2S rather than requiring an external audio decoder or external memory. The device is effective at synthesizing melody patterns, and the resulting output is adequate for a demonstration and course project.

Although the system is limited in scope compared to a modern digital audio player, the project achieves its main goal: demonstrating practical real-time embedded design using STM32 hardware and FreeRTOS. It successfully combines hardware interfacing and software control into a single working device.

16. Conclusion

The Personal MP3 Player project successfully demonstrates the design and implementation of an embedded audio player on the RT-Spark STM32F407 development board. The player incorporates multiple tasks, real-time timers, hardware interrupts, audio synthesis, user controls, and electronic display functionality into one cohesive system.

The design was implemented using PlatformIO, STM32Cube HAL, and FreeRTOS, and it shows how a microcontroller can manage audio, input, display, and scheduling tasks concurrently. The combination of a hardware codec, I2S communication, ADC volume control, LCD display, and user input makes this a strong example of a realistic embedded application.

This project not only illustrates how electronics and software can be integrated, but it also reinforces the importance of efficient control, timing, and resource management in embedded systems. The result is a working demonstration of a compact, educational, and functional personal music player.

17. Recommendations

To improve the project in future iterations, the following enhancements may be considered:

1. Add SD card support for storing actual MP3 or WAV files.
2. Implement a more advanced audio playback system with buffering and streaming from external storage.
3. Add a menu system with more navigation options on the LCD.
4. Improve display graphics and UI design for better readability.
5. Add calibration for more accurate note synthesis and tempo control.
6. Use a more robust audio driver architecture and add error handling for codec failures.
7. Include additional testing for edge cases during rapid key presses and volume adjustments.

These improvements would make the project more versatile while preserving the educational foundation established by the current design.

18. References

1. STMicroelectronics. STM32F407 Datasheet and Reference Manual.
2. RT-Thread RT-Spark Board Documentation.
3. PlatformIO Documentation for STM32 Projects.
4. FreeRTOS Documentation and Real-Time Scheduling Concepts.
5. ES8388 Codec Driver References and Embedded Audio Implementation Notes.
6. Course handouts and project materials provided for the Personal MP3 Player laboratory activity.

19. Appendix A: Project Structure

```text
Personal-MP3-Player-main/
■■■ README.md
■■■ platformio.ini
■■■ fpu_link.py
■■■ include/
■ ■■■ FreeRTOSConfig.h
■ ■■■ lcd.h
■ ■■■ lcd_font.h
■ ■■■ songs.h
■ ■■■ song.h
■ ■■■ song_def.h
■ ■■■ README
■■■ src/
■ ■■■ main.c
■ ■■■ lcd.c
■ ■■■ songs.cpp
■■■ lib/
■ ■■■ FreeRTOS-Kernel/
■■■ ref/
■■■ docs/
■ ■■■ Laboratory Report/
■ ■■■ Laboratory_Report.md
■■■ test/
■■■ .vscode/
```

20. Final Remarks

This laboratory report documents the complete lifecycle of the Personal MP3 Player project, from concept through design, implementation, and functional validation. The work demonstrates that embedded systems can be used not only to solve computational tasks but also to create interactive and practical electronic devices. The project owner, Mary Grace V. Soronio, is the primary contributor and owner of this embedded systems application.